

Chapter 2 · Lesson 1 — Why Next.js + TypeScript

Chapter 2 · Lesson 1 — Why Next.js + TypeScript

Goal: understand the framework decision tree well enough to defend “we used Next.js” in an interview without looking nervous. Beginners pick frameworks by vibe; seniors pick by requirements.

The interview question

“So why did you pick Next.js for this project?”

Bad answer: “Because everyone uses it.” Or: “Because the AI suggested it.”

Good answer: “The app is a static, single-page experience with file-based data. I needed TypeScript out of the box, zero-config deployment, and a way to optionally pre-render at build time. Next.js gave me all three with one command, plus image optimization and routing if I ever extended it. Vite would have worked too — Vite is leaner and faster in dev — but Next gave me Vercel deployment as a free side effect, which mattered.”

That’s the answer this lesson is preparing you to give. We’re going to walk through the decision space.

The React framework landscape (as of 2026)

When you decide to use React, you’re not done. You still have to pick a **build tool** or **meta-framework**. The major options:

Tool	What it is	Best for
Next.js	A meta-framework on top of React. File-based routing, server-side rendering (SSR), static site generation (SSG), API routes, image/font optimization, deployment to Vercel.	Apps that need any combination of: SSR, SEO, multi-page routing, server-side data, deployment ease.
Vite + React	A bare-bones build tool. Just bundles and serves React. No router, no SSR, no nothing extra.	Single-page apps where you want minimal magic. Library or component-collection projects.
Create React App (CRA)	The legacy starter. Officially deprecated by the React team in 2025.	Don't pick this. Migration target only.
Remix	A meta-framework like Next.js, but more loyal to web standards (forms, fetch, etc.).	Form-heavy apps, apps wanting to embrace progressive enhancement.
TanStack Start	A new entrant, "Vite + React + a router + SSR" assembled by hand.	Apps that want Next.js features without Next.js opinions.
Plain HTML + script tag	Just a <code><script src="react.js"></code> and a CDN link.	Demos, single-page widgets, learning.

For our snooker app, we need to satisfy these requirements (from Chapter 1):

- Single-page interaction (one URL, all interaction client-side)
- Static deployment (no backend)
- Data lives in code (TypeScript constants)
- Tiny state (which tab, which selected item)

Vite would work. Next.js would also work. Plain HTML + a script tag would work for a smaller version. CRA is dead. Remix is overkill for a no-form app. TanStack Start is too

new.

So our real choice is **Vite vs Next.js**.

The Vite vs Next.js decision (the honest one)

For an app **this size**, here's the honest tradeoff:

Vite wins on:

- **Dev server startup speed.** Vite boots in ~200ms; Next takes 2-3 seconds.
- **Less framework magic.** What you see in your code is what runs.
- **Smaller bundle by default.** No SSR overhead, no Next runtime.
- **Easier to migrate to a non-React framework later.** Vite is generic.

Next.js wins on:

- **Vercel deployment with literally one click.** Push to GitHub, import on Vercel, done. Vite needs a hosting target you configure (Netlify, Cloudflare Pages, etc.).
- **TypeScript + Tailwind + ESLint pre-wired.** `npx create-next-app` gives you a working project with all of these. With Vite you assemble them yourself.
- **Image and font optimization** out of the box (`next/font`, `next/image`).
- **Routing if you ever expand.** If next year you add a `/leaderboard/[year]` route, Next has it free; Vite needs `react-router-dom`.
- **Server components for free.** Even though we won't use them heavily, the *option* is there.

For this project, **Next.js wins because of deployment ergonomics**. We will probably push this to Vercel, and Next.js + Vercel is the smoothest pipeline available. The cost is a slightly heavier dev server. We'll pay it.

Interview answer template: *"We used Next.js because [main reason] and [secondary reason]. Vite would also have worked — it's lighter — but [main reason] was the tiebreaker."* Replace [main reason] with whichever Next.js feature mattered most for your project. **Always be ready to defend the alternative.** Interviewers respect "I considered X but picked Y because Z" answers, and distrust "I just used what was popular."

Why TypeScript (not plain JavaScript)?

This is a quicker decision. Three reasons.

1. Catches errors at compile time, not at runtime.

The example we'll never get tired of:

```
// JavaScript – silently broken  
const team = teams[0];  
console.log(team.scors.r1); // typo, returns undefined, app shows "undefined"  
  
// TypeScript – error before you save  
const team = teams[0];  
console.log(team.scors.r1); // ERROR: Property 'scors' does not exist on ty
```

You'll make typos. JavaScript will let them ship. TypeScript won't. **Compile-time safety is free; runtime debugging is expensive.** That alone pays for TypeScript a hundred times over.

2. Self-documenting code.

When you write function `calculateTeamScores(team: Team): TeamScores`, anyone reading the code knows the input shape, the output shape, and they can `Cmd+Click` to navigate to either. **Types are documentation that the compiler keeps honest.** Comments drift. Types don't.

3. Editor superpowers.

VS Code, Cursor, Windsurf — every modern editor turns into a god-tier autocomplete machine when you have types. You type `team.` and it shows you `r1`, `r2`, `qf`, `sf`, `final`, ... You don't have to memorize the shape; the editor reminds you.

When NOT to use TypeScript

There's a reasonable counterargument for tiny scripts (under 50 lines), prototypes you'll throw away, and learning exercises where the type ceremony obscures the lesson. For *anything* shipped to production, multi-file, or maintained by more than one person — TypeScript wins.

Interview answer template: *“Compile-time errors, self-documenting types, and editor support. The cost is a small upfront learning curve and slightly more verbose code, but for any project bigger than a 50-line script the win is overwhelming.”*

Why Tailwind CSS (and why we barely use it)

This is the one part of the stack that has a small surprise. The codebase uses **inline styles** (e.g. `style={{ background: '#0F5132' }}`) more than Tailwind classes. Then why is Tailwind installed at all?

Three reasons:

1. To set up `globals.css` properly.

`app/globals.css` has these three Tailwind directives at the top:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

`@tailwind base` is the part that matters most: it injects Tailwind’s CSS reset (Preflight). That’s a curated set of “make all browsers behave the same” rules — `* { box-sizing: border-box }`, `h1 { font-size: inherit }`, etc. **Without a reset, every browser styles your app slightly differently.** With Tailwind installed we get the reset for free, and we don’t even have to use any classes.

2. To leave the door open for future Tailwind use.

If you later want to convert one component from inline styles to Tailwind classes, the toolchain’s already set up. No re-config needed.

3. So `next/font` and other Next.js features that integrate with Tailwind work cleanly.

Some Next plugins assume Tailwind is present. Having it costs ~5KB of CSS and zero developer time.

Why inline styles instead of Tailwind classes?

The codebase uses dynamic, computed styles based on team colors, gradients with multiple custom values, and conditional logic that would produce very long class strings if rendered with Tailwind:

```
// Inline (what the codebase uses)
<div style={{
  background: `linear-gradient(135deg, ${team.color} 0%, ${team.accent} 100%`,
  boxShadow: `0 8px 32px ${team.color}50`,
  borderRadius: 16,
}}>

// Tailwind (what it'd look like with arbitrary values)
<div className="rounded-2xl"
  style={{ background: `linear-gradient(135deg, ${team.color} 0%, ${team.accent} 100%`,
    boxShadow: `0 8px 32px ${team.color}50` }}>
```

Tailwind's strength is **static styling repeated across many elements** (buttons, headings, cards). When every visual is computed from runtime props (team colors), Tailwind doesn't help — and inline styles read more cleanly. **Right tool, right job.**

In an interview: *"We use Tailwind for the reset and global setup; inline styles for component-level dynamic theming. If a piece of styling becomes static and repeated, we'd convert it to Tailwind classes."* That's a defensible answer.

What about CSS Modules, styled-components, emotion, vanilla-extract?

Quick tour of the alternatives:

- **CSS Modules** (.module.css) — built into Next.js. Locally-scoped class names. Great for static styling. We could have used them; we chose inline because of the dynamic theming. Both are valid for production.
- **styled-components / emotion** — CSS-in-JS libraries. Powerful but add bundle size and runtime overhead. Falling out of favor (2023+) because of bundle size and SSR pain. Don't reach for them on new projects.
- **vanilla-extract** — a typed, zero-runtime CSS-in-TS library. Newer, niche. Worth knowing about; not necessary here.

For our app: **inline styles + Tailwind reset** is the simplest, lightest approach that satisfies our constraints. **Senior engineers prefer the simplest thing that works.**

Why we don't need a state management library (Redux, Zustand, Jotai)

We have **two pieces of top-level state**: `activeTab` and `selectedTeam`. That's it.

For two pieces of state, `useState` in the orchestrator component plus prop drilling is **the right answer**. Reach for Redux when:

- A dozen-plus components share the same data,
- The state is updated from many places,
- You need time-travel debugging or middleware (e.g. logging actions),
- You're building a Figma-tier app with operational transforms.

We're building a snooker fantasy league. None of the above apply. **Picking Redux for this app would be over-engineering — and interviewers can tell.** A senior says *"the smallest thing that works."* If the requirements grow, you migrate. You don't pre-migrate.

Interview tip. When asked *"how do you manage state?"*, your answer should always start with the size of the app. *"For our small app, we use `useState` in the parent component and prop drilling. If state coordination grew complex, we'd reach for Zustand first because it's the lightest option that scales beyond `useState`."* That answer signals you've thought about it; it signals you don't reach for libraries you don't need; it signals you know the next step if needed.

Why Recharts (when it's time for charts)

Foreshadowing Chapter 6, but worth noting now.

We use **Recharts** for the analytics tab. Alternatives:

- **Chart.js + react-chartjs-2** — older, larger, imperative API. Works but feels last-decade.
- **Victory** — declarative like Recharts, slightly more flexible, slightly bigger.
- **D3** — the underlying library. Massively powerful but you write more code.
- **Visx** — Airbnb's collection of low-level D3-React components. Power-user choice.
- **Nivo** — beautiful, declarative, larger bundle.

Recharts is the **default React charting library**. It's:

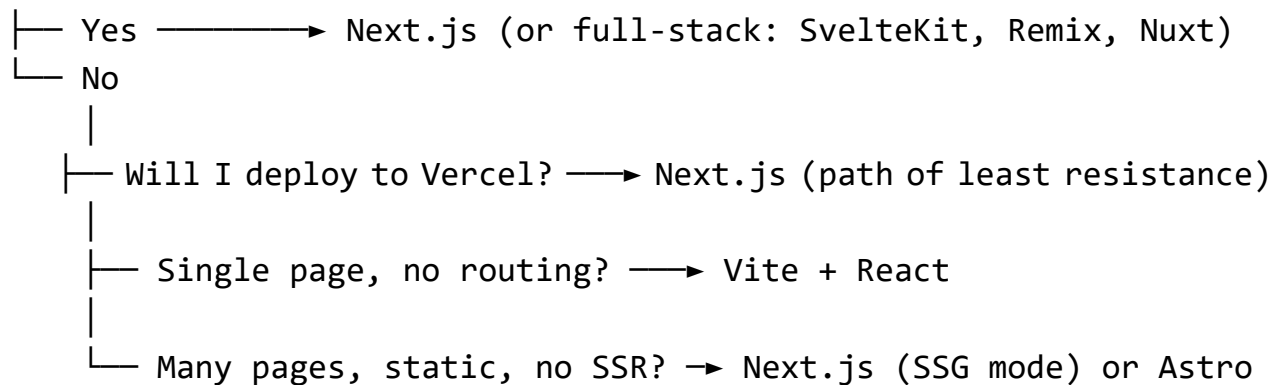
- Declarative (you describe what you want, not how to draw it).
- Built on D3 internally (so you get D3's correctness without D3's complexity).
- Tree-shakeable (only the chart types you import end up in the bundle).
- TypeScript-friendly out of the box.

We're not building Bloomberg Terminal. Recharts is enough. **Don't reach for D3 directly until Recharts can't do what you need.**

The decision tree, in one sketch

If you were starting from scratch tomorrow and had to choose, here's the flowchart in your head:

Do I need a backend / API?



Do I need TypeScript? —————> Yes. Always. (Unless < 50 LOC throwaway)

Do I need a state library? —————> Only if useState + prop drilling can't handle it.
Try Zustand before Redux.

Do I need a styling library? —————> Tailwind. Or inline styles. Don't pick CSS-in-JS in 2026.

Do I need charts? —————> Recharts. Don't reach for D3 until Recharts fails.

Memorize that flowchart. It's not infallible, but it's the senior default. You'll be wrong sometimes; that's fine. Defending the wrong default is easier than defending no default.

What this lesson did NOT cover

- **Server vs client components in detail** (Chapter 7).
- **The exact package .json contents** (next lesson).
- **How Next.js bundles your code under the hood** (advanced — out of scope).
- **Why Server Actions exist** (Next.js 14+ feature; we don't use them).

These are all worth a Google later if you're curious. For now, the framework decisions are made. Next lesson: we type the commands and create the project.

Vibe prompt you would have used (to make the framework decision)

"I'm building a single-page React app for a fantasy sports league. ~250 hand-typed data records, 5 tabs, no backend, no auth, no real-time, deployment to Vercel. Should I use Next.js or Vite + React? Lay out the tradeoffs honestly. Don't just say 'use the popular one.'"

The "don't just say use the popular one" line is what gets the LLM to actually weigh trade-offs instead of giving you the most upvoted Stack Overflow answer.

CHECK YOURSELF

1. **A friend says "but Next.js has SSR, isn't that wasted on a static page?"** Compose a one-paragraph defense of using Next.js for this project anyway.
2. **Your interviewer says "Vite is faster in dev — why didn't you use it?"** Give the honest two-sentence answer.
3. **Why does the project have Tailwind installed even though most styles are inline?** State three concrete reasons.
4. **Tomorrow you start a new project: a 200-product e-commerce site with checkout. Walk through the framework decision tree out loud.**
5. **Tomorrow you start a different new project: a Figma-style collaborative whiteboard. Different answer? Why?**

When you're ready, head to [02-scaffolding-the-project.md](#) and we'll start typing commands.